Introduction - LV Distribution Planning and Validation

This application is an interactive planning tool designed to support the early-stage design and validation of low-voltage (LV) electricity distribution networks for rural electrification and mini-grid systems. The tool provides a structured workflow that allows users to move from spatial demand data, such as building locations and estimated electricity consumption, to a candidate distribution network topology and then evaluate whether that network is electrically feasible under realistic operating conditions. The platform is implemented as a Streamlit application and combines geographic heuristics, electrical modeling, and optimization techniques to provide a fast yet technically meaningful screening environment for distribution network design.

Intended Workflow

The workflow is organized into three sequential steps. In the first step, the tool synthesizes a **candidate grid topology using spatial heuristics**. Starting from building locations and optional road network data, the algorithm generates candidate pole locations, assigns users to poles based on distance and capacity constraints, and constructs a radial backbone network using graph-based methods such as minimum spanning trees. Additional support poles are inserted where necessary to respect maximum span lengths, and multiple consistency checks ensure that the resulting network is geometrically coherent and fully connected. The outcome of this stage is a candidate LV distribution layout with associated infrastructure metrics such as total line length, pole counts, and service coverage.

In the second step, the **candidate network is electrically validated through a power-flow analysis**. The tool aggregates electricity demand at each pole based on building metadata and load profile assumptions, assigns electrical parameters to each line, and constructs a simplified power system model using the **PyPSA framework**. A single-snapshot power flow is then executed to evaluate voltage levels and line loading across the network. This validation step allows users to quickly identify electrical feasibility issues such as excessive voltage drops or overloaded lines, which commonly arise in long radial LV networks with significant distributed demand.

The third step extends the validation stage by introducing a **reinforcement optimization procedure**. In this phase, the topology of the network remains fixed, but the thermal capacity of the distribution lines can be increased through optimization. Using PyPSA's linear optimization capabilities, the model identifies the minimum reinforcement required, expressed as increased line capacity, to resolve thermal overloads while meeting the demand assigned to each node. This provides a simple yet powerful mechanism to estimate the additional infrastructure investment required to make a candidate topology electrically feasible.

Scope and Limitations

The tool is primarily intended for **planning and pre-feasibility analysis**, where rapid exploration of alternative network configurations is more valuable than detailed engineering precision. It enables planners, researchers, and practitioners to compare spatial layouts, understand the electrical implications of network design choices, and estimate distribution infrastructure requirements before undertaking more detailed engineering studies.

However, several limitations should be recognized. The topology generation stage relies on heuristic spatial algorithms rather than full electrical optimization, meaning that the resulting layout is not guaranteed to be globally optimal. The electrical validation currently relies on a single power-flow snapshot rather than time-series simulations, so it does not capture daily or seasonal variability in demand. Furthermore, the reinforcement optimization focuses only on increasing line thermal capacity and does not currently modify conductor impedance, change conductor types, or redesign the network topology. As a result, voltage

violations may persist in networks with very long feeders or high resistance lines, even after thermal reinforcement. Consequently, the tool should be understood as a **decision-support platform for early-stage grid planning**, rather than a substitute for detailed distribution system engineering.

Workflow overview

The app currently has three stages:

1. **Topology synthesis** (Page 1)
    - Build poles + MST network from users and optional roads.
2. **Validation / PF** (Page 2)
    - Load topology (session or external files), aggregate demand, assign electrical assumptions/line params, run single-snapshot PF.
3. **Reinforcement optimization** (Page 2 Step 3 section)
    - On fixed topology + fixed load snapshot + fixed slack, optimize line capacity upgrades.

Input groups by stage:

- Topology:
    - Users file, optional roads file, heuristic and coverage sliders/checkboxes.
- Validation/PF:
    - Topology source files (if external), demand CSVs, PF settings, line-parameter mode + CSVs, PF run controls.
- Reinforcement:
    - Upgrade scope mode, cost fallback, max upgrade factor, optional emergency shedding, optional solver name, run trigger.

Topology inputs

Users file (.xlsx or .gpkg)

Stage: Topology synthesis
UI location: Page 1 → “Upload connection data”
Input type: File upload (dist_users_file)
Required/optional: Required
Accepted values / format:

- .xlsx: must include columns latitude and longitude (exact lowercase in current code path check).
- .gpkg: geometry read via geopandas; no mandatory attribute columns.
  Expected units:
- .xlsx: lat/lon in degrees (assumed EPSG:4326).

- .gpkg: projected CRS expected/converted to target CRS.
  Default value: None
  Used by code
  in: pages/ui_sections/topology_sections.py → pages/1_Grid_Topology.py → core/distribution_service.run_low_voltage() → core/distribution_io.load_and_transform_data()
  Internal role: Source building points for pole association and coverage decisions. Building IDs are derived from dataframe index.
  Impact on results: Determines settlement geometry, number of buildings, associations, and final network structure.
  Important constraints / caveats:

- If file unreadable or empty: ValueError("Users file could not be loaded or is empty.").
- Missing geometry or still geographic after transform triggers errors in run_low_voltage.
- For .xlsx, if latitude/longitude not found exactly, loader returns None (then run fails).
  Related outputs: gdf_buildings_4326, served/unserved sets, associations, poles, MST edges.

Roads file (.gpkg)

Stage: Topology synthesis
UI location: Page 1 → shown only when “Follow roads” selected
Input type: File upload (dist_roads_file)
Required/optional: Optional globally; required if mode = “Follow roads”
Accepted values / format: .gpkg only (in topology page UI)
Expected units: Projected CRS meters (converted to target CRS if needed)
Default value: None
Used by code
in: render_upload_section → run_low_voltage → load_and_transform_data → collect_sampled_points
Internal role: Provides candidate pole sampling points along road geometries.
Impact on results: Strongly affects pole placement pattern; with roads, poles are first sampled along roads before filling gaps with additional poles.
Important constraints / caveats:

- If “Follow roads” and file missing: UI blocks run with error.
- collect_sampled_points accesses geometry.coords; this is robust for LineString, potentially fragile for certain non-line geometries (doc gap).
  Related outputs: Candidate poles, final poles, MST geometry.

Pole placement strategy

Stage: Topology synthesis
UI location: Page 1 → “Pole placement strategy”
Input type: Radio
Required/optional: Required selection
Accepted values / format:

- Follow roads (requires roads file)
- Free placement
  Expected units: N/A
  Default value: Follow roads (index 0)
  Used by code in: render_upload_section; conditional check in pages/1_Grid_Topology.py

Internal role: Controls whether road file is expected and whether road-sampled poles are created.
Impact on results: Switches initial pole candidate generation method.
Important constraints / caveats: In free mode, roads file is ignored.
Related outputs: Pole distribution and resulting MST.

Road pole spacing (road_pole_spacing_m)

Stage: Topology synthesis
UI location: Page 1 → "Heuristic pole placement and customer association"
Input type: Slider int
Required/optional: Required parameter
Accepted values / format: 10–200, step 5
Expected units: meters
Default value: config.settings.DEFAULT_SAMPLING_DISTANCE_M = 40
Used by code in: run_low_voltage(...
sampling_distance=...) → collect_sampled_points / sample_points_along_line
Internal role: Distance between sampled road candidate poles.
Impact on results: Lower spacing -> denser candidate poles -> potentially shorter user-pole distances and different associations.
Important constraints / caveats: Only meaningful in road-following mode.
Related outputs: Pole count, service drops, backbone geometry.

Max user-pole connection radius (max_user_connection_radius_m)

Stage: Topology synthesis
UI location: Page 1 parameter section
Input type: Slider int
Required/optional: Required
Accepted values / format: 10–200, step 5
Expected units: meters
Default value: config.settings.DEFAULT_USER_DISTANCE_M = 35
Used by code in: run_low_voltage(...
user_distance=...) → associate_buildings_to_poles and place_poles_for_unassociated_buildings; also promotion pass after densification
Internal role: Maximum distance for assignment/association logic.
Impact on results: Larger radius usually reduces number of poles and unassociated buildings; smaller radius increases new pole creation and possible unserved clusters.
Important constraints / caveats: Applied in multiple sub-steps; not just one pass.
Related outputs: Associations, pole counts, served/unserved.

Max users per pole (max_users_per_pole)

Stage: Topology synthesis
UI location: Page 1 parameter section
Input type: Slider int
Required/optional: Required
Accepted values / format: 1–100, step 1
Expected units: count
Default value: config.settings.DEFAULT_MAX_ASSOCIATIONS = 16
Used by code in: run_low_voltage(... max_associations=...) → association + unassociated pole placement + promotion capacity checks

Internal role: Capacity cap for building assignments per pole.
Impact on results: Lower cap increases pole count and potentially service length.
Important constraints / caveats: Hard cap used repeatedly in algorithm.
Related outputs: Associations, serving poles count.

Max LV span between poles (max_pole_span_m)

Stage: Topology synthesis
UI location: Page 1 → "Network span control (post-processing)"
Input type: Slider int
Required/optional: Required
Accepted values / format: min = current road spacing, max 150, step 5
Expected units: meters
Default value: 80
Used by code in: run_low_voltage(... max_pole_span_m=...) → densify_mst_edges
Internal role: Densifies long MST edges by inserting support poles.
Impact on results: Smaller value increases support poles, shorter individual spans.
Important constraints / caveats:

- If <=0 would disable densification at backend level, but UI always gives positive.
- Later cleanup merges very-close poles (FINAL_MERGE_TOL_M=10.0) and asserts no ultra-short edges remain.
  Related outputs: Support poles, MST edges, total poles.

Allow isolated unserved (allow_unserved_isolated)

Stage: Topology synthesis
UI location: Page 1 → "Coverage behaviour"
Input type: Checkbox
Required/optional: Optional
Accepted values / format: bool
Expected units: N/A
Default value: False
Used by code in: run_low_voltage(...
allow_unserved_isolated=...) → place_poles_for_unassociated_buildings
Internal role: Enables selective coverage mode.
Impact on results: Can leave small isolated clusters unserved (standalone candidates).
Important constraints / caveats: If false, model attempts full coverage and sets unserved set empty.
Related outputs: num_unserved, red points in map.

Minimum cluster size (min_cluster_size)

Stage: Topology synthesis
UI location: Page 1, shown only when previous checkbox is true
Input type: Slider int
Required/optional: Conditional
Accepted values / format: 1–10, step 1
Expected units: buildings (count)
Default value: 2 when shown; forced to 1 when checkbox false
Used by code in: run_low_voltage(... min_cluster_size=...) → place_poles_for_unassociated_buildings
Internal role: Cluster threshold below which clusters remain unserved in selective mode.

Impact on results: Higher threshold tends to increase unserved candidates.
Important constraints / caveats: Ignored effectively when selective mode disabled (forced 1).
Related outputs: Served vs unserved counts.

Run / Clear buttons

Stage: Topology synthesis
UI location: Page 1 action row
Input type: Buttons
Required/optional: Trigger actions
Accepted values / format: click events
Expected units: N/A
Default value: not clicked
Used by code in: pages/1_Grid_Topology.py
Internal role: Execute run or clear session topology/validation state.
Impact on results: Clear resets both topology and validation state (clear_topology(... clear_validation=True)).
Important constraints / caveats: Clear invalidates downstream validation/reinforcement context.
Related outputs: Session domains and all rendered outputs.

Hidden/internal topology assumptions

Stage: Topology synthesis
UI location: Not exposed directly
Input type: Internal constants/logic
Required/optional: Always active
Accepted values / format:

- TARGET_CRS = 32633 (config/settings.py)
- Dedup tolerance pre-MST: 0.75 m
- Final merge tolerance: 10.0 m
  Expected units: meters / EPSG code
  Default value: fixed in code
  Used by code in: distribution_io, distribution_service
  Internal role: CRS normalization and geometry cleanup stability.
  Impact on results: Can materially alter final node count and edge lengths.
  Important constraints / caveats: Not user-configurable in UI.
  Related outputs: All topology geometries/metrics.

Validation / PF inputs

Topology source mode

Stage: Validation / PF
UI location: Page 2 → “Grid Topology”
Input type: Radio
Required/optional: Required selection
Accepted values / format:

- Use Page 1 session results
- Load external files
  Expected units: N/A
  Default value: external (index 1)
  Used by code in: render_topology_source_section + branch logic in pages/2_Grid_Validation.py
  Internal role: Chooses data ingestion path.
  Impact on results: Session path uses in-memory topology; external path validates uploaded schemas.
  Important constraints / caveats: If session selected but no topology exists, user must run Page 1 first.
  Related outputs: ValidationInputs.mode (session or external).

External nodes file

Stage: Validation / PF
UI location: Page 2 external mode
Input type: File upload
Required/optional: Required in external mode
Accepted values / format: .geojson, .json, .gpkg via read_vector
Expected units: geographic CRS expected eventually in mapping; must have CRS for nodes (adapter enforces)
Default value: None
Used by code in: read_vector → validate_external_topology → validation_inputs_from_external_payload
Internal role: Bus/pole geometry and IDs.
Impact on results: Defines buses and centering for PF/map.
Important constraints / caveats:

- Must contain pole_id or id column.
- Nodes CRS must be defined (else adapter raises error).
  Related outputs: gdf_nodes_4326, pole list for slack selection.

External edges file

Stage: Validation / PF
UI location: Page 2 external mode
Input type: File upload
Required/optional: Required in external mode
Accepted values / format: .geojson, .json, .gpkg
Expected units: geometry in lat/lon or convertible; endpoints are IDs
Default value: None
Used by code in: read_vector → validate_external_topology
Internal role: Electrical connectivity edges for PF.
Impact on results: Determines network graph and line lengths.
Important constraints / caveats:

- Must include endpoint columns from accepted aliases:
  - u/source: source, bus0, from, u
  - v/target: target, bus1, to, v
- Self-loops removed, edges with unknown endpoint IDs dropped, undirected duplicates deduped.

- For catalog line-params mode in external topology, a line_id column is required by UI logic.
  Related outputs: gdf_edges_4326, edge endpoint columns, PF lines.

External associations CSV

Stage: Validation / PF
UI location: Page 2 external mode
Input type: CSV upload
Required/optional: Required in external mode
Accepted values / format: CSV with pole/building mapping
Expected units: IDs (non-unit)
Default value: None
Used by code in: pd.read_csv + validate_external_topology + _get_associations_from_validation_inputs
Internal role: Maps buildings to poles for demand aggregation.
Impact on results: Controls where load is injected.
Important constraints / caveats:

- Column aliases accepted: pole = pole_id or pole; building = building_id or building.
- pole_id coerced to numeric/int; non-numeric causes error.
  Related outputs: associations_df, pole load aggregation.

Building metadata CSV

Stage: Validation / PF demand
UI location: Page 2 → "Demand inputs"
Input type: CSV upload
Required/optional: Required to aggregate loads
Accepted values / format: CSV with building and category (+ optional weight)
Expected units: weight is dimensionless multiplier
Default value: none (weight fallback per row=1.0 if missing/NaN)
Used by code in: read_building_metadata_csv → aggregate_pole_loads
Internal role: Converts building associations to category-weighted pole demand.
Impact on results: Sets relative/absolute pole loads.
Important constraints / caveats:

- Required columns via aliases:
  - building: building_id or building
  - category: category or user_category or type
- Optional weight aliases: weight, multiplier, scale.
- If no match with associations by building_id, aggregation fails.
  Related outputs: pole_loads_kW timeseries.

Category profiles CSV

Stage: Validation / PF demand
UI location: Page 2 → "Demand inputs"
Input type: CSV upload
Required/optional: Required to aggregate loads
Accepted values / format: Wide table: hour + one column per category

Expected units: kW per building (as documented by code comments/UI text)
Default value: None
Used by code in: read_category_profiles_csv → aggregate_pole_loads
Internal role: Time profiles per category used in matrix multiplication with building weights.
Impact on results: Defines hour-by-hour system demand and PF snapshot values.
Important constraints / caveats:

- Hour column aliases accepted: hour, t, time, snapshot.
- Hours must be numeric and unique.
- At least one category column required.
- Categories in metadata must exist in profile columns (else error).
  Related outputs: pole_loads_kW, hour sliders.

Bubble scaling mode

Stage: Validation / PF visualization
UI location: Page 2 demand controls
Input type: Selectbox
Required/optional: Optional visualization choice
Accepted values / format: Absolute / Relative
Expected units: N/A
Default value: Absolute
Used by code in: render_demand_controls + map function make_map_lv_with_load_bubbles
Internal role: Chooses fixed yearly max vs per-hour max for bubble radius scaling.
Impact on results: Visual only, does not change PF.
Important constraints / caveats: None
Related outputs: load bubble map.

Hour for visualization (pf_vis_hour_slider)

Stage: Validation / PF visualization + stored runtime state
UI location: Page 2 demand controls
Input type: Slider int
Required/optional: Required after demand aggregation
Accepted values / format: min/max from profile hour index
Expected units: hour index
Default value: peak total-load hour
Used by code in: update_validation_load_state
Internal role: Chooses displayed hour and pole_load_dict stored in validation inputs.
Impact on results: Affects map/load preview and slack suggestion weighting context.
Important constraints / caveats: PF run uses separate PF hour slider later.
Related outputs: load map, captions.

Highlight pole controls

Stage: Validation visualization
UI location: load map section
Input type: checkbox + selectbox
Required/optional: Optional
Accepted values / format: bool + pole ID or None

Expected units: pole ID
Default value: disabled; default highlighted pole is max-load pole if available
Used by code in: render_load_visualization → make_map_lv_with_load_bubbles
Internal role: map convenience only
Impact on results: Visual only.
Important constraints / caveats: None
Related outputs: load map.

Slack / plant pole

Stage: PF setup
UI location: Page 2 → "Power flow setup"
Input type: Selectbox of available pole IDs
Required/optional: Required
Accepted values / format: integer pole IDs from nodes
Expected units: ID
Default value: suggested by load-weighted centroid heuristic
Used by code in: update_validation_pf_settings → PFScenarioParams.slack_pole_id → run_snapshot
Internal role: Single generator/slack bus location.
Impact on results: Strong impact on voltage profile and line flows.
Important constraints / caveats: If slack ID not in nodes, PF errors.
Related outputs: bus voltages, loading, maps.

Voltage limits (v_min_pu, v_max_pu)

Stage: PF setup
UI location: Page 2 setup row
Input type: Number inputs
Required/optional: Required
Accepted values / format:

- min: 0.70–1.00
- max: 1.00–1.30
  Expected units: per unit
  Default value: 0.90 / 1.10
  Used by code in: PF params + violation flags in outputs/maps
  Internal role: Compliance thresholds; also validated min < max.
  Impact on results: Changes violation counts/feasibility reporting (not physics itself).
  Important constraints / caveats: Invalid ordering stops execution.
  Related outputs: summary violations, map red buses.

Load power factor (pf_load)

Stage: PF setup
UI location: Page 2 setup row
Input type: Number input
Required/optional: Required
Accepted values / format: 0.50–1.00
Expected units: dimensionless pf
Default value: 0.95
Used by code in: PFScenarioParams → _infer_q_from_pf in runner

Internal role: Converts P to reactive Q for loads.
Impact on results: Lower pf increases Q and usually worsens voltage drop/loading.
Important constraints / caveats: backend enforces 0 < pf_load <= 1.
Related outputs: voltages, line apparent power/loading.

Voltage base convention (v_base_mode)

Stage: PF setup
UI location: Page 2 setup
Input type: Selectbox
Required/optional: Required
Accepted values / format:

- 3-phase LV (0.4 kV line-to-line)
- Per-phase equivalent (0.230 kV L-N)
  Expected units: mode mapping to nominal kV
  Default value: UI select index 0, but one-time migration sets session default to per-phase if key absent
  Used by code in: maps to v_nom_kv:
- 3-phase -> 0.4
- per-phase -> 0.4/sqrt(3)
  and load scaling factor (3.0 if v_nom_kv < 0.3)
  Internal role: Defines electrical base in PyPSA network and load scaling convention.
  Impact on results: Significant impact on impedance base and voltage profile.
  Important constraints / caveats: persistence now fixed via one-time migration flag.
  Related outputs: PF results and debug.

Line parameter mode

Stage: PF line assumptions
UI location: Page 2 expander “Line parameters”
Input type: Radio
Required/optional: Required choice
Accepted values / format:

- global
- catalog
- catalog_overrides
  Expected units: mode selector
  Default value: global
  Used by code in: build_line_params_for_edges and state update
  Internal role: Chooses parameter assignment strategy for each edge.
  Impact on results: Directly affects r/x/s_nom used in PF.
  Important constraints / caveats: external catalog modes require line_id on external edges.
  Related outputs: resolved line params preview, PF flows.

Global R/X/S_nom

Stage: PF line assumptions
UI location: same expander
Input type: Number inputs
Required/optional: Required in all modes (also serve as defaults)
Accepted values / format:

- R: 0.0001–5.0 (ohm/km)
- X: 0.0001–5.0 (ohm/km)
- S_nom: 1.0–1000.0 (kVA)
  Expected units: ohm/km, ohm/km, kVA
  Default value: 0.642 / 0.083 / 100
  Used by code in: global mode directly; also as default_params in catalog modes
  Internal role: electrical line model inputs.
  Impact on results: very high sensitivity on voltage and loading.
  Important constraints / caveats: backend rejects nonphysical values (r>0, x>=0, s_nom>0).
  Related outputs: PF metrics/results.

line_types.csv

Stage: PF line assumptions (catalog modes)
UI location: line parameter expander (when mode != global)
Input type: CSV upload
Required/optional: Required in catalog and catalog_overrides
Accepted values / format: required columns:
line_type, r_ohm_per_km, x_ohm_per_km, s_nom_kva
Expected units: ohm/km, ohm/km, kVA
Default value: none
Used by code in: read_line_types_csv / validate_line_types / build_line_params_for_edges
Internal role: Cable catalog.
Impact on results: Defines per-line electrical parameters through type mapping.
Important constraints / caveats:

- unique nonblank line_type
- numeric columns required
- physical constraints enforced
  Related outputs: resolved line params table, PF behavior.

Default line type (catalog modes)

Stage: PF line assumptions
UI location: selectbox appears after line_types.csv load
Input type: Selectbox
Required/optional: Required in catalog modes
Accepted values / format: one of loaded line_type values
Expected units: N/A
Default value: first option or prior state
Used by code in: build_line_params_for_edges(default_line_type=...)
Internal role: fallback for edges lacking metadata row.

Impact on results: Sets baseline type for unmatched lines.
Important constraints / caveats: must exist in line types set.
Related outputs: resolved line params.

lines_metadata.csv (optional)

Stage: PF line assumptions
UI location: line parameter expander (catalog modes)
Input type: CSV upload
Required/optional: Optional
Accepted values / format: must include line_id, optional:
line_type, r_ohm_per_km_override, x_ohm_per_km_override, s_nom_kva_override
Expected units: same as line params
Default value: if absent, synthetic metadata with only line IDs (fallback to default line type)
Used by code in: read_lines_metadata_csv / build_line_params_for_edges
Internal role: per-line type assignment and optional overrides.
Impact on results: selectively modifies line electrical parameters.
Important constraints / caveats:

- no duplicate/blank line_id
- no extra line IDs not in edge table
- override constraints validated (r>0, x>=0, s_nom>0).
  Related outputs: resolved line params and PF outputs.

PF min edge length filter (pf_min_len_m)

Stage: PF run controls
UI location: “Run Power Flow (PyPSA)” controls
Input type: Number input
Required/optional: Required
Accepted values / format: 0.0–50.0, step 1
Expected units: meters
Default value: 5.0 m
Used by code in: converted to km and passed to run_snapshot(min_len_km=...)
Internal role: drops very short segments to improve numerical stability.
Impact on results: can remove tiny edges; too high can disconnect/over-prune network.
Important constraints / caveats: if all edges removed -> PF error.
Related outputs: PF feasibility/results/debug stats.

PF system base (pf_sn_mva)

Stage: PF run controls
UI location: PF controls
Input type: Number input
Required/optional: Required
Accepted values / format: 0.01–100.0
Expected units: MVA
Default value: 0.1 (one-time session migration if missing)
Used by code in: run_snapshot(sn_mva=...)
Internal role: network base power for per-unit conversion.
Impact on results: affects impedance base and numerical conditioning.

Important constraints / caveats: too small/large can destabilize behavior.
Related outputs: PF voltages/loading/debug.

PF fail on nonsense (pf_fail_on_nonsense)

Stage: PF run controls
UI location: PF controls
Input type: Checkbox
Required/optional: Optional
Accepted values / format: bool
Expected units: N/A
Default value: True
Used by code in: run_snapshot(check_nonsense=...)
Internal role: if PyPSA returns non-physical voltages, runner may trigger radial fallback solver logic.
Impact on results: controls strictness/fallback behavior.
Important constraints / caveats: false may allow suspect outputs to pass farther.
Related outputs: debug + solver path.

PF hour slider (pf_run_hour_slider)

Stage: PF run controls
UI location: PF controls
Input type: Slider
Required/optional: Required to run PF
Accepted values / format: min/max hour from aggregated loads
Expected units: hour index
Default value: peak total-load hour
Used by code in: selects pole_loads_kW.loc[pf_hour]
Internal role: snapshot hour for PF run and stored ValidationResult.hour.
Impact on results: directly changes load snapshot, hence all PF outputs.
Important constraints / caveats: independent from visualization hour slider.
Related outputs: PF results/map.

Run PF button

Stage: PF run
UI location: PF controls
Input type: Button
Required/optional: Trigger
Accepted values / format: click event
Expected units: N/A
Default value: not clicked
Used by code in: pages/2_Grid_Validation.py run branch
Internal role: executes PF and stores typed result.
Impact on results: creates/updates current PF result.
Important constraints / caveats: Reinforcement step depends on existing PF result.
Related outputs: ValidationResult.

Reinforcement optimization inputs

Run even if clean (rf_run_even_if_clean)

Stage: Reinforcement
UI location: Step 3 settings expander
Input type: Checkbox
Required/optional: Optional
Accepted values / format: bool
Expected units: N/A
Default value: auto = whether PF has current violations
Used by code in: Page 2 run guard before calling optimizer
Internal role: blocks unnecessary optimization when no violations unless user forces it.
Impact on results: determines whether optimization can run in clean case.
Important constraints / caveats: if false and no violations, button only shows info message.
Related outputs: whether reinforcement result is produced.

Reinforcement scope (rf_selection_mode)

Stage: Reinforcement
UI location: Step 3 settings
Input type: Selectbox
Required/optional: Required
Accepted values / format:

- all_lines
- overloaded_only
- violating_feeder_path
  (mapped from user-friendly labels)
  Expected units: mode
  Default value: all lines
  Used by code in: _collect_candidate_line_ids
  Internal role: chooses which existing edges are allowed to expand (s_nom_extendable=True).
  Impact on results: narrows/widens optimization decision set.
  Important constraints / caveats:
- feeder-path mode uses shortest path from slack to buses with voltage violations.
- if non-all mode yields empty set, code falls back to overloaded_only.
  Related outputs: reinforced lines set and cost.

Cost fallback (rf_cost_per_km_per_kva)

Stage: Reinforcement economics
UI location: Step 3 settings
Input type: Number input
Required/optional: Required
Accepted values / format: 0.0001–100.0
Expected units: currency / (km·kVA)
Default value: 0.08
Used by code in: line capital_cost assignment and estimated cost report

Internal role: objective coefficient for line expansion cost.
Impact on results: influences which lines are upgraded and objective value/cost.
Important constraints / caveats: must be >0; example line_types.csv cost_per_m is currently ignored by reinforcement code.
Related outputs: total reinforcement cost, line-level estimated cost.

Max upgrade factor (rf_max_upgrade_factor)

Stage: Reinforcement constraints
UI location: Step 3 settings
Input type: Number input
Required/optional: Required
Accepted values / format: 1.0–50.0
Expected units: multiplier
Default value: 4.0
Used by code in: sets s_nom_max = old_s_nom * factor
Internal role: cap on each line capacity expansion.
Impact on results: can prevent infeasible/too-large upgrades.
Important constraints / caveats: must be >=1.
Related outputs: delta_s_nom_kva, feasibility.

Emergency load shedding toggle (rf_allow_shedding)

Stage: Reinforcement feasibility
UI location: Step 3 settings
Input type: Checkbox
Required/optional: Optional
Accepted values / format: bool
Expected units: N/A
Default value: False
Used by code in: if true, adds high-cost generators at load buses (carrier="load_shedding")
Internal role: emergency infeasibility relief.
Impact on results: can avoid optimization infeasibility by allowing curtailed served load representation.
Important constraints / caveats: disabled by default; fixed topology/load philosophy otherwise maintained.
Related outputs: optimization status/objective.

Shedding penalty (rf_shedding_penalty)

Stage: Reinforcement feasibility/economics
UI location: Step 3 settings (enabled only if shedding checked)
Input type: Number input
Required/optional: Conditional
Accepted values / format: 1,000 to 1e9
Expected units: currency/MWh
Default value: 100,000
Used by code in: marginal_cost of shedding generators
Internal role: discourages load shedding.
Impact on results: higher penalty pushes line reinforcement over shedding.
Important constraints / caveats: ignored when shedding toggle off.
Related outputs: objective and selected upgrades.

Solver name (rf_solver_name)

Stage: Reinforcement solve config
UI location: Step 3 settings
Input type: Text input
Required/optional: Optional
Accepted values / format: string or blank
Expected units: solver identifier
Default value: blank (None)
Used by code in: passed to n.optimize(solver_name=...) if provided
Internal role: solver selection override.
Impact on results: performance/availability; potentially different solve behavior.
Important constraints / caveats: no UI validation of solver presence.
Related outputs: optimization status.

Optimize button

Stage: Reinforcement execution
UI location: Step 3 settings
Input type: Button
Required/optional: Trigger
Accepted values / format: click
Expected units: N/A
Default value: not clicked
Used by code in: pages/2_Grid_Validation.py reinforcement branch
Internal role: runs run_reinforcement_optimization(...) on PF hour snapshot.
Impact on results: generates ReinforcementResult.
Important constraints / caveats: uses current PF run settings (pf_min_len_m, pf_sn_mva, pf_fail_on_nonsense) and the PF result hour.
Related outputs: pre/post summary, reinforced lines table/map, CSV export.

Cross-file consistency requirements

1. **building_id linkage**
    - Associations (from topology or external) must match building_metadata.csv building IDs as strings after coercion.
    - No matches -> demand aggregation fails.
2. **Category linkage**
    - building_metadata.category values must exist as column names in category_profiles.csv.
    - Missing category columns -> error.
3. **Nodes/edges ID consistency (external topology)**
    - Nodes must include pole_id or id.
    - Edges endpoints must reference valid node IDs after numeric coercion.

- Invalid endpoint edges are dropped during validation.

4. **Associations pole IDs**
    - Associations pole IDs must be numeric; non-numeric values error.
    - In topology stage, associations are tightly checked against final MST graph/pole table.
5. **Line IDs**
    - Catalog modes require stable line_id.
    - For session topology, line IDs are generated (L0, L1, ...).
    - For external catalog mode, edges file must include line_id.
6. **Session vs external mode differences**
    - Session mode uses mst_edges_pole_ids as PF edge source.
    - External mode uses uploaded edges + inferred endpoint columns.
    - Session mode always treated as having explicit line IDs for catalog checks (via generated edges path).
7. **CRS expectations**
    - Topology users .xlsx assumed EPSG:4326 input and reprojected.
    - Topology gpkg users/roads reprojected to TARGET_CRS (or assumed target if missing CRS).
    - Validation external nodes must have CRS; edges CRS may be missing (assumed 4326 in adapter).

File schema reference tables

Topology users file (.xlsx)

column	required	type	meaning	units	notes
latitude	yes	numeric	building latitude	degrees	exact name required by loader check
longitude	yes	numeric	building longitude	degrees	exact name required by loader check
others	no	any	carried through dataframe	n/a	not required by topology algorithm

Topology users file (.gpkg)

field	required	type	meaning	units	notes
geometry	yes	point-like expected	building locations	CRS units	IDs derived from row index; no mandatory attribute columns

Roads file (.gpkg)

field	required	type	meaning	units	notes
geometry	yes	line geometry expected	candidate pole sampling support	CRS units	algorithm assumes geometry.coords availability in sampling function

External topology nodes file (.geojson/.json/.gpkg)

column	required	type	meaning	units	notes
pole_id or id	yes	numeric/int-coercible	node ID	n/a	one required
geometry	yes	point or geometry with representative point	node location	CRS	nodes CRS required by adapter

External topology edges file (.geojson/.json/.gpkg)

column	required	type	meaning	units	notes
endpoint u alias	yes	int-coercible	from node ID	n/a	one of source,bus0,from,u
endpoint v alias	yes	int-coercible	to node ID	n/a	one of target,bus1,to,v
line_id	conditional	string	stable edge ID	n/a	required for catalog modes in external path
geometry	recommended	LineString/MultiLineString	used for map/length	CRS	PF length estimated from geometry endpoints

Associations CSV (external)

column	required	type	meaning	units	notes
pole_id or pole	yes	numeric/int-coercible	serving pole ID	n/a	required
building_id or building	yes	string-like	building key	n/a	required

building_metadata.csv

column	required	type	meaning	units	notes
building_id (or building)	yes	string-like	building key	n/a	must match associations IDs after coercion
category (or user_category/type)	yes	string	demand category	n/a	must exist in profile columns
weight (or multiplier/scale)	no	numeric	per-building multiplier	dimensionless	defaults/fills to 1.0

category_profiles.csv

column	required	type	meaning	units	notes
hour (or t/time/snapshot)	yes	numeric int	snapshot index	hour index	unique required
category columns	at least one	numeric	kW per building for that category	kW	names must cover metadata categories

line_types.csv

column	required	type	meaning	units	notes
line_type	yes	unique string	cable type name	n/a	nonblank

column	required	type	meaning	units	notes
r_ohm_per_km	yes	numeric > 0	resistance	ohm/km	validated
x_ohm_per_km	yes	numeric >= 0	reactance	ohm/km	validated
s_nom_kva	yes	numeric > 0	thermal rating	kVA	validated
other columns	no	any	ignored by current merge logic	n/a	e.g. sample cost_per_m currently unused

lines_metadata.csv

column	required	type	meaning	units	notes
line_id	yes	unique string	edge key	n/a	must exist in current edges set
line_type	no	string	explicit type assignment	n/a	blank -> NA
r_ohm_per_km_override	no	numeric > 0	per-line R override	ohm/km	only in catalog_overrides mode
x_ohm_per_km_override	no	numeric >= 0	per-line X override	ohm/km	only in catalog_overrides mode
s_nom_kva_override	no	numeric > 0	per-line thermal override	kVA	only in catalog_overrides mode

---

Parameter sensitivity / practical interpretation

- **Road pole spacing:** lower spacing densifies candidate poles; tends to reduce service drops but can increase pole count.
- **Max user-pole radius:** higher radius increases assignment reach; often fewer poles, potentially longer service connections.
- **Max users per pole:** lower value forces more poles.

- **Max pole span:** lower value inserts more support poles and reduces long-span artifacts.
- **Selective coverage + min cluster:** higher cluster threshold leaves more isolated users unserved.
- **Demand weight and category profile values:** directly scale pole demand; high weights/categories dominate loading and voltage drops.
- **Slack pole choice:** central slack often improves downstream voltages and reduces worst loading.
- **PF power factor:** lower PF increases Q and worsens voltage/loading.
- **Voltage base mode:** per-phase equivalent changes nominal base and load scaling; strongly affects reported p.u. voltages.
- **R/X/S_nom:** primary electrical sensitivity knobs; R/X affect voltage drop and flows, S_nom affects thermal violations.
- **PF min edge length filter:** useful for stability; too high may remove meaningful edges.
- **PF sn_mva:** changes per-unit base/conditioning; extreme values can distort behavior.
- **Reinforcement max upgrade factor:** limits feasible upgrades per line.
- **Reinforcement cost per (km·kVA):** drives upgrade selection trade-offs.
- **Reinforcement scope mode:** controls decision-space size and what can be reinforced.

Defaults and fallback behavior

UI defaults

- Topology:
    - road spacing 40, user radius 35, max users/pole 16, max span 80
    - selective coverage disabled
- Validation:
    - topology source defaults to external upload mode
    - bubble scaling defaults Absolute
    - v_min 0.90, v_max 1.10, pf_load 0.95
    - v_base selectbox index 0 (3-phase), but one-time migration may initialize session key to per-phase if missing
    - line mode defaults global; global params 0.642 / 0.083 / 100
    - PF controls: min edge 5 m, sn_mva 0.1 (session migration), fail_on_nonsense true
- Reinforcement:
    - scope all lines
    - cost fallback 0.08

    - max factor 4.0
    - emergency shedding off
    - shedding penalty 100000
    - solver blank

Backend defaults/fallbacks

- TARGET_CRS=32633
- Topology dedup/merge tolerances: 0.75 m and 10 m.
- Building metadata weight missing/invalid -> 1.0.
- Category profile nonnumeric values -> coerced then filled 0.0.
- Missing line metadata file -> fallback metadata with only line IDs.
- Global line params fallback if not using catalog.
- run_snapshot defaults:
    - min_len_km=0.005 if not passed
    - sn_mva=1.0 default argument (UI passes explicit value)
- Reinforcement non-all mode fallback:
    - if selected candidate lines empty -> retry overloaded-only selection.

Silent assumptions users should know

- Users .xlsx header names must include latitude and longitude exactly in current loader logic.
- Topology building IDs come from row index (or preserved dataframe index), not required input column.
- Example line_types.csv has cost_per_m, but current reinforcement optimizer does not consume it.

Validation checks and common failure modes

Topology stage

- Missing users file.
- “Follow roads” selected without roads file.
- Users file unsupported/empty/missing geometry.
- Users still geographic CRS after transform.
- No poles could be placed.
- Internal topology consistency checks:
    - associations refer to missing poles

- serving poles disconnected from MST
- ultra-short final edges remain after cleanup

External topology validation

- Empty nodes/edges/associations.
- Missing node ID columns (pole_id/id).
- Missing edge endpoint columns.
- Non-numeric association pole IDs.
- No valid numeric node IDs.
- Nodes missing CRS (adapter error).

Demand aggregation

- Missing required columns in metadata/profiles.
- No association-building ID join matches.
- Categories in metadata not found in profile columns.
- Non-numeric/duplicate hour issues.

Line parameter setup

- Missing/invalid line_types.csv columns or values.
- Missing/invalid lines_metadata.csv line_id.
- Metadata includes line_id not present in edge table.
- Catalog mode without line_id in external edges.
- Final merged line params missing required fields.

PF run

- Invalid voltage bounds or pf.
- Slack ID not in nodes.
- Load poles not connected by any edge (orphan load).
- All edges filtered out by min length/used poles.
- Missing resolved line parameters for some edges.
- Multiple electrical islands violate single-slack assumption.
- PyPSA non-convergence/nonsense voltages may trigger fallback solver path or fail.

Reinforcement optimization

- Invalid reinforcement settings (cost <=0, factor <1).
- Baseline network unavailable.

- Solver not available (if user sets unsupported solver name).
- Optimization may reduce overloads but voltage violations can remain (capacity-only upgrade mode).

Example minimal valid datasets

Minimal run: topology step only

Use:

- examples/users.xlsx (already valid; contains User, latitude, longitude columns)
- optionally examples/roads.gpkg
- no extra CSV needed.

Minimal parameter set:

- defaults are sufficient.

Minimal run: validation using session topology

After running Page 1:

- use session topology mode in Page 2
- upload:
    - examples/building_metadata.csv
    - examples/category_profiles.csv
- keep line mode global with defaults.

This is the minimum to run PF from session outputs.

Minimal run: validation using external topology

Need three files:

1. nodes (.geojson/.gpkg) with pole_id or id + geometry + CRS
2. edges (.geojson/.gpkg) with endpoint columns (u/v or aliases) + geometry
3. associations CSV with building_id and pole_id (or aliases)

Then upload metadata and profiles CSV as above.

If you want catalog mode:

- edges must carry line_id
- plus line_types.csv (required)
- optional lines_metadata.csv.

Minimal run: reinforcement step

Prerequisites:

- successful PF result exists in Page 2.
- then Step 3 can run with all defaults (scope all lines, cost 0.08, factor 4).

Output includes:

- pre/post summary
- reinforced lines table and CSV
- post-optimization map (reinforced edges highlighted).

Code index

Main input-definition/consumption points:

- Topology UI and controls:
  - pages/ui_sections/topology_sections.py
  - pages/1_Grid_Topology.py
- Topology loaders/algorithms:
  - core/distribution_io.py
  - core/distribution_service.py
  - core/distribution_algos.py
- Validation UI and controls:
  - pages/ui_sections/validation_sections.py
  - pages/2_Grid_Validation.py
- Validation file parsing and schemas:
  - core/powerflow_io.py (read_vector, read_building_metadata_csv, read_category_profiles_csv)
  - core/powerflow_validation.py (validate_external_topology, aggregate_pole_loads)
- PF model:
  - core/powerflow_network.py (PFScenarioParams, PFTopologyBundle, PyPSAPowerFlowRunner.run_snapshot)
- Line parameter catalogs:
  - core/line_params.py
- Reinforcement:
  - core/powerflow_reinforcement.py

    - Step-3 hooks in pages/2_Grid_Validation.py and pages/ui_sections/validation_sections.py
- State/contracts/adapters:
    - core/contracts.py
    - core/pipeline_state.py
    - core/pipeline_adapters.py
- Defaults:
    - config/settings.py
- Examples/tests:
    - examples/*
    - tests/test_line_params_merge.py
    - tests/test_powerflow_reinforcement.py